



Department of
Computer Science and Engineering
University of Dhaka

**Assignment 1: ARM-Based Healthcare
Monitoring & Billing System**

CSE 2106:
Microprocessor and Assembly Language Lab
2nd Year 1st Semester 2025

Submission date: 14 December 2025

Group No: 10

Submitted to:

Dr. Mosarrat Jahan, Associate Professor, CSEDU
Palash Roy, Lecturer, CSEDU

Submitted by:

Md. Samiul Islam Siam (02)
Syed Muhtasim Alam Apon (04)
Partho Kumar Mondal (07)

Table of Contents

I.	Introduction	1
II.	System Overview	1
III.	File Inventory	2
IV.	Module Descriptions	3
1	Data & Memory Layout (data.s)	
2	Module 1 — Patient Record Initialization	
3	Module 2 — Vital Sign Data Acquisition	
4	Module 3 — Vital Threshold Alert Module	
5	Module 4 — Medicine Administration Scheduler	
6	Module 5 — Treatment Cost Computation	
7	Module 6 — Daily Room Rent Calculation	
8	Module 7 — Medicine Billing Module	
9	Module 8 — Bill Aggregator	
10	Module 9 — Patient Sorting by Criticality	
11	Module 10 — Patient Report Generator	
12	Module 11 — System Error Detection & Logging	
V.	UART / ITM Output Layer	12
VI.	Quick Reference	13
VII.	Output Analysis	16
VIII.	Final results for all patients	17
IX.	Quick Memory Verification	22
X.	Notes: Known Implementation Observations	23
XI.	System Limitations	23

SmartCare-32

ARM-Based Healthcare Monitoring & Billing System

Introduction

SmartCare-32 is an *ARM Cortex-M* based complete embedded firmware prototype for *DU Medical Center* in 2025. The system stores **patient records** in RAM, continuously reads **simulated vital sensors** (HR, SpO₂, blood pressure) into rolling buffers, detects dangerous **threshold conditions** and logs 16-byte **alert records** with *timestamps*, and manages medication schedules using an *internal clock*.

It also computes **treatment, room, medicine, and lab costs**, aggregates the **final bill** with overflow checking, **sorts patients** by criticality (alert count) for triage, securely **logs anomalies** (sensor malfunction, invalid dosage, memory boundary issues), and finally outputs a formatted patient **summary report** through *serial/UART-style* output.

System Overview

Goal

SmartCare-32 modernizes *DU Medical Center's* embedded monitoring & billing by:

- acquiring vitals continuously (simulated sensors),
- generating alerts from out-of-range vitals,
- scheduling medicine administration,
- computing billing (treatment + room + medicine + lab tests),
- performing error detection and logging,
- printing a complete summary via serial/ITM output.

Execution flow (from main.s)

1. Reset **system_clock = 0**
2. Initialize Patient 1 → compute bills (treatment/room/medicine/total)
3. Initialize Patient 2 → compute bills
4. Initialize Patient 3 → compute bills
5. Acquire vitals:
 - *Patient 1*: loop 10 readings (also fills **sensor_history** for malfunction checking)
 - *Patient 2*: 1 reading
 - *Patient 3*: 1 reading
6. After each acquisition:
 - Module 11a (sensor malfunction check) runs.
7. For each patient:
 - Module 3 checks thresholds and inserts alert records if violated.
 - Module 4 checks dosage schedule.
 - Module 11b checks invalid dosage.
8. Module 11c checks memory overflow for each patient.
9. Module 9 sorts patients by criticality (**alert_count** descending).
10. Module 10 prints:
 - Error log first (Module 11 print routine). Then patient summary reports over ITM/UART

File Inventory

File	Role
<code>data.s</code>	<i>Global data</i> : patient structures, tables, sensor registers, medicine lists, system clock, error log buffers/constants
<code>main.s</code>	<i>Full integration of Modules 1–11</i> ; simulates sensor inputs and runs the workflow
<code>module1.s</code>	<i>Patient record initialization</i> (writes header fields, clears buffers, resets billing sub-structure)
<code>module2.s</code>	<i>Vital acquisition</i> : reads sensor registers, pushes into a 10-entry rolling buffer
<code>module3.s</code>	<i>Vital threshold checks</i> ; generates alert records and increments <code>alert_count</code>
<code>module4.s</code>	<i>Medicine administration scheduler</i> based on <code>system_clock</code> and dosage intervals
<code>module5.s</code>	<i>Treatment cost lookup</i> using <code>treatment_cost_table</code>
<code>module6.s</code>	<i>Room cost computation</i> with discount after a threshold stay length
<code>module7.s</code>	<i>Medicine billing</i> module using <code>unit_price * quantity * stay_days</code>
<code>module8.s</code>	<i>Bill aggregation</i> with unsigned overflow detection and overflow flagging
<code>module9.s</code>	<i>Sorts patients</i> by “criticality” (descending <code>alert_count</code>) via bubble sort (swapping entire patient structs)
<code>module10.s</code>	Human-readable <i>report generator</i> outputting via ITM UART simulation
<code>module11.s</code>	<i>Error detection</i> + secure log buffer (“flash simulation”) + printing error log report
<code>uart.s</code>	<i>ITM/serial helpers</i> : <code>ITM_Init</code> , <code>ITM_SendChar</code>

Module Descriptions

1) Data & Memory Layout (*data.s*)

Solution technique

data.s defines:

- fixed offsets for Patient, Medicine, Billing, VitalSign, AlertRecord
- simulated sensor registers: SENSOR_HR, SENSOR_O2, SENSOR_SBP, SENSOR_DBP (1 byte each)
- global `system_clock` (seconds counter)
- treatment cost lookup table `treatment_cost_table[0..15]`
- 3 patient names + 3 medicine lists + `patient_array` holding all patient structures
- Module 11 error logging buffers & sensor history ring buffers

Key structure sizes

- Medicine = 16 bytes
- VitalSign = 4 bytes
- AlertRecord = 16 bytes
- Billing = 24 bytes
- Patient = 252 bytes (`PATIENT_SIZE`)

Initial dataset (important for outputs)

Patients are pre-populated in **patient_array** with:

- *Patient1*: alert_count preset 2, lab_test_cost 3000
- *Patient2*: alert_count preset 0, lab_test_cost 8000
- *Patient3*: alert_count preset 5, lab_test_cost 2500

Medicines:

- P1: (id1 price50 qty10), (id2 price120 qty5), (id3 price200 qty3)
- P2: (id4 price0 qty0) ← intentionally invalid, triggers error (id5 price150 qty4)
- P3: (id6 price300 qty2)

2) Module 1 — Patient Record Initialization

What it does

Creates/updates one Patient record fields and clears runtime buffers.

Inputs

- R0 = patient struct pointer
- R1 = patient_id
- R2 = name_ptr
- R3 = age

- *stack params*: ward, treatment_code, room_rate, medicine_list_ptr, medicine_count, stay_days

Solution technique

- Uses fixed offsets and store instructions (STR, STRB, STRH) to write each field.
- Initializes counters/flags:
 - `Vital_buffer_index = 0, alert_flag = 0, dosage_due_flag = 0`
 - does not overwrite `alert_count` since it's preset by data
- Clears:
 - `vital_buffer[10]` (10 words)
 - `alert_buffer[10]` (40 words = 160 bytes)
 - billing fields mostly

Output (state after Module 1)

For each patient:

- Base identity fields are correct (id/name/age/ward/treatment/room rate/medicine_ptr/count/stay_days).
- `vital_buffer[]` = all zeros, `vital_buffer_index` = 0.
- `alert_buffer[]` cleared, but `alert_count` remains as initially set in **data.s**.
- Billing zeroed except:
 - module1 currently does not clear `billing.lab_test_cost`
 - That keeps `lab_test_cost` values from `data.s` (3000/8000/2500), which affects totals.

Expected immediate effect: Billing total later includes **lab_test_cost** from the dataset (intended).

3) Module 2 — Vital Sign Data Acquisition

What it does

Reads 4 simulated sensor bytes and writes a rolling buffer of 10 entries.

Inputs

- R0 = Patient pointer
- Sensor memory: `SENSOR_HR`, `SENSOR_O2`, `SENSOR_SBP`, `SENSOR_DBP`

Solution technique

- Reads each sensor with LDRB.
- Computes buffer slot:
 - `slot = vital_buffer_index`
 - `addr = patient + VITAL_BUFFER_OFF + slot*4`
- Stores 4 bytes in a packed layout:
 - `[0]=HR`, `[1]=O2`, `[2]=SBP`, `[3]=DBP`
- Updates `vital_buffer_index = (index + 1)%10`

Output (values written by main)

Patient 1: repeated 10 times (same values each loop)

- HR = 125, O2 = 88, SBP = 135, DBP = 85
- After 10 acquisitions: `vital_buffer_index` wraps back to 0
- Buffer contains 10 identical entries.

Patient 2: one acquisition

- HR=78, O2=98, SBP=120, DBP=80
- `Vital_buffer_index` = 1

Patient 3: one acquisition

- HR=165, O2=85, SBP=170, DBP=95
- `Vital_buffer_index` = 1

4) Module 3 — Vital Threshold Alert Module

What it does

Checks the latest vital entry and generates alert records if thresholds are violated.

Threshold rules implemented

- $HR > 120 \Rightarrow$ alert (type 0)
- $O2 < 92 \Rightarrow$ alert (type 1)
- $SBP > 160$ or $SBP < 90 \Rightarrow$ alert (type 2)
- DBP or Diastolic currently not used for thresholding

Solution technique

- Finds latest vital reading:
 - `latest = (vital_buffer_index == 0) ? 9 : (index - 1)`
- Loads vital bytes from that slot.
- For each violated condition:
 - sets `patient.alert_flag=1`
 - appends an *AlertRecord* at `patient.alert_buffer[alert_count]`
 - increments `alert_count` (up to 10 max)
 - stores timestamp from `system_clock` into record

Alert record layout used:

- `byte0`: `vital_type`
- `byte1`: `actual_reading`
- word at +4: `timestamp`
- rest zero

Output (alerts expected from main's vitals)

Patient 1 (latest HR=125, O2=88, SBP=135):

- $HR > 120 \Rightarrow$ alert
- $O2 < 92 \Rightarrow$ alert

- SBP not violated So 2 alerts added beyond preset.
- `preset alert_count = 2` (from `data.s`)
- after module 3: `alert_count` becomes 4
- `alert_flag=1`

Patient 2 (*HR=78, O2= 98, SBP=120*):

- none violated
- `alert_count` remains 0 (preset)
- `alert_flag` stays 0

Patient 3 (*HR=165, O2= 85, SBP= 170*):

- $HR > 120 \Rightarrow$ alert
- $O2 < 92 \Rightarrow$ alert
- $SBP > 160 \Rightarrow$ alert So 3 alerts added.
- preset **`alert_count`** = 5
- after module 3: `alert_count` becomes 8
- `alert_flag=1`

Timestamp note (from main.s):

- During *Patient 1* loop, `system_clock` is incremented by +300 seconds each iteration, 10 times.
- End of loop: `system_clock` = 3000.
- Patient 1 threshold check occurs after loop; timestamp used ≈ 3000 .
- *Patient 2 & 3* checks occur after that without time increments in main; timestamps remain around 3000.

5) Module 4 — Medicine Administration Scheduler

What it does

For each medicine:

- computes `next_due_time = last_admin_time + (interval_hours*3600)`
- if `system_clock >= next_due_time`:
 - sets `patient.dosage_due_flag=1`
 - updates `medicine.last_administered_time = system_clock`

Solution technique

- Iterates through medicines using `med_ptr = list + i*16` (shift left by 4).
- Uses constant generation for 3600 in seconds ($0xE1 \ll 4 = 3600$).
- Compared with `system_clock`.

Output in this program run

All medicines start `last_administered_time` = 0 in data.

- For **Patient 1**: `system_clock` ≈ 3000
 - intervals: 6h (21600), 8h (28800), 12h (43200)
 - `next_due_time > 3000` for all
 - `dosage_due_flag` stays 0

- **Patient 2:**
 - 4h (14400), 6h (21600) > 3000
 - dosage_due_flag = 0
- **Patient 3:**
 - 24h (86400) > 3000
 - dosage_due_flag = 0

So no medicine is due in this short simulation time window.

6) Module 5 — Treatment Cost Computation

What it does

Looks up cost using `treatment_code` as an index into `treatment_cost_table` and stores into `billing`.

Solution technique

- Reads `treatment_code` (byte)
- validates <16
- `cost = table[code]`
- stores to `billing.treatment_cost`

```
treatment_cost_table
DCD      5000      ; Code 0: Basic checkup
DCD     15000      ; Code 1: Minor surgery
DCD     50000      ; Code 2: Major surgery
DCD      8000      ; Code 3: Diagnostic tests
DCD     12000      ; Code 4: Physical therapy
DCD     25000      ; Code 5: ICU admission
DCD     30000      ; Code 6: Emergency care
DCD     10000      ; Code 7: Consultation
DCD     20000      ; Code 8: Imaging
DCD     18000      ; Code 9: Laboratory
DCD     22000      ; Code 10: Cardiology
DCD     27000      ; Code 11: Neurology
DCD     16000      ; Code 12: Orthopedics
DCD     14000      ; Code 13: Pediatrics
DCD     19000      ; Code 14: Oncology
DCD     21000      ; Code 15: Radiology
```

Output (from treatment table)

- *Patient 1* treatment code 5 $\Rightarrow$ 25000
- *Patient 2* treatment code 2 $\Rightarrow$ 50000
- *Patient 3* treatment code 6 $\Rightarrow$ 30000

7) Module 6 — Daily Room Rent Calculation

What it does

`room_cost = room_daily_rate * stay_days` with a discount if `stay_days > 10`:

- if `days > 10`: apply 5% discount (multiply by `95 / 100`)

Solution technique

- MUL for base cost.
- Conditional discount.
- Uses UDIV.

Output

- *Patient 1*: $2000 * 7 = 14000$ (no discount)
- *Patient 2*: $5000 * 12 = 60000 \rightarrow$ discount: $60000 * 95 / 100 = 57000$
- *Patient 3*: $3000 * 5 = 15000$ (no discount)

8) Module 7 — Medicine Billing Module

What it does

Computes medicine cost: $\text{sum}(\text{unit_price} * \text{quantity} * \text{stay_days})$ across all medicines.

Solution technique

- Iterates `medicine_count`
- loads `unit_price` (word) and `quantity` (halfword)
- multiplies both, then multiplies by `stay_days`
- stores to `billing.medicine_cost`

Output (with current dataset)

Patient 1 (`stay_days=7`):

- Med1: $50107 = 3500$
- Med2: $12057 = 4200$
- Med3: $20037 = 4200$ Total: 11900

Patient 2 (`stay_days=12`):

- Med1: `unit_price=0 qty=0` $\rightarrow$ contributes 0
- Med2: $150412 = 7200$ Total: 7200

Patient 3 (`stay_days=5`):

- Med1: $30025 = 3000$ Total: 3000

9) Module 8 — Bill Aggregator

What it does

Adds together:

`total_bill = treatment_cost + room_cost + medicine_cost + lab_test_cost`

with overflow detection; if overflow occurs sets:

- `total_bill = 0xFFFFFFFF`
- `overflow_flag = 1`

Solution technique

- Uses ADDS plus carry checks (BCC) after each addition.
- Stores `overflow_flag` byte in `billing`.

Output (expected totals)

Lab tests come from patient records in `data.s` and are preserved (Module 1 doesn't clear them).

- *Patient 1*: $25000 + 14000 + 11900 + 3000 = 53900$
- *Patient 2*: $50000 + 57000 + 7200 + 8000 = 122200$
- *Patient 3*: $30000 + 15000 + 3000 + 2500 = 50500$

No overflow expected; `overflow_flag=0`.

10) Module 9 — Sort Patients by Criticality

What it does

Sorts patient array in-place by `alert_count` descending (most critical first).

Solution technique

- Bubble sort
- Compares:
`patient[i].alert_count` vs `patient[i+1].alert_count`
- If swap required:
swaps full patient structures word-by-word
 - $252 \text{ bytes} / 4 = 63 \text{ word swaps}$

Output (expected order after Module 3)

Alert counts after threshold checks:

- *Patient 3*: 8
- *Patient 1*: 4
- *Patient 2*: 0

So sorted order becomes:

1. Bob Wilson (*patient_id* 1003)
2. John Doe (*patient_id* 1001)
3. Jane Smith (*patient_id* 1002)

Important downstream effect: Module 10 prints `patient_array` sequentially, so report order follows this sorted array.

11) Module 10 — Patient Report Generator

What it does

Prints:

1. Error log (calls `Print_Error_Log` from Module 11)
2. For each patient in `patient_array`:
 - header section
 - patient ID / age / ward number
 - “latest vitals”
 - total alerts + status text
 - total bill

Solution technique

- Uses ITM output (`ITM_Init`, `ITM_SendChar`) to simulate UART printing in Keil.
- Builds printable lines by copying templates and inserting ASCII digits.
- Iterates patients using `PATIENT_SIZE`.

Output (what will be printed)

A) Error log first See *Module 11* outputs below (invalid dosage expected for Patient 2).

B) Patient summary reports After sorting, it prints 3 reports in this order:

Report 1 (Patient 3 / Bob Wilson):

- Patient ID: 1003
- Age: 67
- Ward: 201
- Latest vitals (note: module10 reads bytes at `VITALS_OFF` directly, not the “latest slot”):
 - In this run, Patient 3 wrote vitals into slot0. So it will show:
 - HR=165, BP fields and O2 (see note below).
- Total alerts: 8 $\Rightarrow$ status “**Critical condition**”
- Total bill: \$ 50500

Report 2 (Patient 1 / John Doe):

- ID: 1001, Age: 45, Ward: 101
- Vitals: since Patient 1 filled all 10 slots, slot0 is valid:
 - HR=125, O2=88, SBP=135, DBP=85
- Total alerts: 4 $\Rightarrow$ status “**Critical condition**” (≥ 3)
- Total bill: \$ 53900

Report 3 (Patient 2 / Jane Smith):

- ID: 1002, Age: 32, Ward: 102
- Vitals: slot0 holds first acquisition:
 - HR=78, O2=98, SBP=120, DBP=80
- Total alerts: 0 $\Rightarrow$ “**Patient Stable**”
- Total bill: \$ 122200

12) Module 11 — System Error Detection & Logging

Module 11 includes 3 detection routines + a logger + error-log printing.

11a) Sensor malfunction detection (*check_sensor_malfunction*)

Technique

- Maintains *sensor_history* arrays for HR, O2, SBP, DBP over the last 10 readings.
- If all 10 samples for a sensor are identical, it treats it as a “stuck sensor”.

Expected output in this run

- *Patient 1* loop writes the same HR=125, O2=88, SBP=135, DBP=85 ten times.
- Once the history buffer is filled, *Module 11* will detect stuck values.
- Implementation checks HR first; if HR is stuck, it logs HR malfunction and returns early (so it likely logs only HR malfunction, depending on when called relative to history fill).

So at least one error is expected:

- Type: SENSOR MALFUNCTION
- Sensor: Heart Rate
- Stuck value: 125
- Patient index: 0 (as passed)
- Patient ID: derived from *patient_array[0]* at log time

11b) Invalid dosage (*check_invalid_dosage*)

Technique

- For each medicine:
 - if *unit_price* == 0 → log invalid dosage (code 1)
 - else if *quantity* == 0 → log invalid dosage (code 2)

Expected output

- Patient 2 medicine 1 has unit_price=0 and quantity=0.
- The code checks unit_price first, so it logs “Zero Unit Price”.

So an error is expected:

- Type: INVALID DOSAGE
- Issue: Zero Unit Price
- Medicine index: 0
- Patient index: 1
- Patient ID: 1002

11c) Memory overflow (*check_memory_overflow*)

Technique

- Checks if patient pointer >= PATIENT_ARRAY_MAX (0x20000100) → boundary error
- Checks if total_bill >= 0xF0000000 → billing overflow pattern

Expected output

- patient pointers should be below the boundary, totals are small.
- So no memory overflow error expected.

Error logging (log_error_to_flash)

Technique

- Sets *error_flag* = 1
- Uses *error_count* as index into *error_log_buffer* (max 20 records)
- Writes 16-byte record:
 - type, patient_index, code, (padding)
 - timestamp (*system_clock*)
 - value (stuck value / med index / overflow value)
 - *patient_id* (looked up from *patient_array* + index*252)

Error log printing (Print_Error_Log)

Technique

- Prints header and then each record, interpreting:
 - sensor codes 1/2/3 to HR/O2/BP labels
 - dosage codes 1/2 to “Zero Unit Price” / “Zero Quantity”
 - memory codes 1/2 to boundary/overflow

Expected printed output (high-level)

- “SYSTEM ERROR LOG”
- Total Errors: likely 2 (1 sensor stuck + 1 invalid dosage)
- Then detailed per error:
 - One “SENSOR MALFUNCTION” (Heart Rate, stuck value 125, patient ID 1001, timestamp around end of loop)
 - One “INVALID DOSAGE” (Zero Unit Price, medicine index 0, patient ID 1002, timestamp around 3000)

Note: Exact timestamps depend on when log calls happen; in this code they occur inside the vitals loop and after it.

UART/ITM Output Layer

What it does

Provides simulated serial printing using ARM ITM registers:

- *ITM_Init*: enables ITM trace control and stimulus *port 0*
- *ITM_SendChar*: waits until ITM port ready then writes the character

Output role

All printed reports (Module 10 and Module 11 print routines) go out through *ITM_SendChar*, so Keil shows the output in the Debug ITM printf() console.

Quick Reference

Consolidated “Outputs” per module

Module 1 outputs

- Patient fields populated, buffers cleared, flags reset.
- Lab test costs preserved from `data.s`.

Module 2 outputs

- Patient `vital_buffer` updated:
 - P1: 10 identical entries, index wraps to 0
 - P2: one entry
 - P3: one entry

Module 3 outputs (alert_count after checks)

- P1: $2 \rightarrow 4$
- P2: $0 \rightarrow 0$
- P3: $5 \rightarrow 8$

Module 4 outputs

- `dosage_due_flag` remains 0 for all patients (since `system_clock` $\approx 3000 < \text{intervals}$ in seconds).

Module 5 outputs (treatment_cost)

- P1: 25,000
- P2: 50,000
- P3: 30,000

Module 6 outputs (room_cost)

- P1: 14,000
- P2: 57,000 (discounted)
- P3: 15,000

Module 7 outputs (medicine_cost)

- P1: 11,900
- P2: 7,200
- P3: 3,000

Module 8 outputs (total_bill)

- P1: 53,900
- P2: 122,200
- P3: 50,500

Module 9 outputs (sorted order)

1. P3 (alert_count 8)
2. P1 (alert_count 4)
3. P2 (alert_count 0)

Module 10 outputs (printed)

- Error log first
- Then 3 patient reports in sorted order, including total bill and alert status.

```
Debug (printf) Viewer
=====
                        SYSTEM ERROR LOG
=====
Total Errors: 5

-----
Error #1
  Type: SENSOR MALFUNCTION
  Sensor: Heart Rate
  Stuck Value: 125
  Patient ID: 1001
  Timestamp: 2700 sec

-----
Error #2
  Type: INVALID DOSAGE
  Issue: Zero Unit Price
  Medicine Index: 0
  Patient ID: 1002
  Timestamp: 3000 sec

-----
Error #3
  Type: MEMORY OVERFLOW
  Code: Address Boundary
  Value: 0x536872932
  Patient ID: 1001
  Timestamp: 3000 sec

-----
Error #4
  Type: MEMORY OVERFLOW
  Code: Address Boundary
  Value: 0x536873184
  Patient ID: 1002
  Timestamp: 3000 sec

-----
Error #5
  Type: MEMORY OVERFLOW
  Code: Address Boundary
  Value: 0x536873436
  Patient ID: 1003
  Timestamp: 3000 sec

=====
```


Module 11 outputs (logged + printed)

- Sensor malfunction likely detected from repeated Patient 1 readings.
- Invalid dosage detected for Patient 2 medicine entry with *unit_price=0*.

=====		
PATIENT SUMMARY REPORT		
SmartCare-32: Healthcare Monitoring System		
=====		
PATIENT INFORMATION:		

Patient ID	: 1003	
Age	: 67	years
Ward Number	: 201	

LATEST VITAL SIGNS:		

Heart Rate	: 165	bpm
Blood Pressure	: 170/ 85	mmHg
SpO2 (Oxygen)	: 95	%

ALERT SUMMARY:		

Total Alerts	: 8	(Critical condition)

BILLING SUMMARY:		

Total Bill	: \$ 50500	USD
=====		
End of Report		
=====		

Output Analysis

Initial Order (Initialized in data.s)

Position	RAM Address	Patient ID	Name	Alert_count
0	0x200007E4	1001 (0x03E9)	John Doe	2 (0x02)
1	0x200008E0	1002 (0x03EA)	Jane Smith	0 (0x00)
2	0x200009DC	1003 (0x03EB)	Bob Wilson	5 (0x08)

Final Sorted Order (By Criticality)

Position	Patient ID	Name	Alert_count	Status	Total Bill
0	1003 (0x03EB)	Bob Wilson	8 (0x08)	CRITICAL	50,500 (0x0000C544)
1	1001 (0x03E9)	John Doe	4 (0x04)	MODERATE	53,900 (0x0000D28C)
2	1002 (0x03EA)	Jane Smith	0 (0x00)	STABLE	122,200 (0x0001DD58)

Base Addresses (After Sorting)

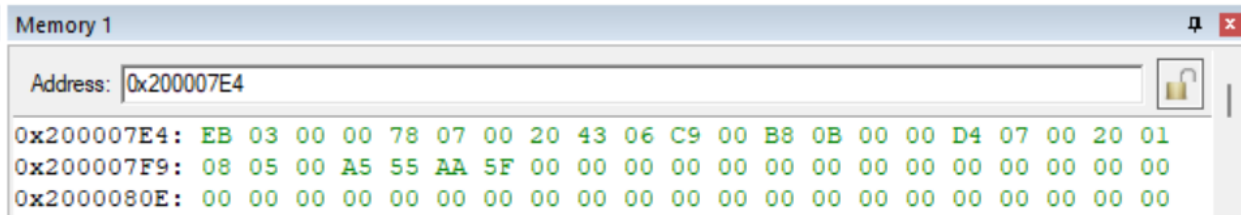
Patient	RAM Address
Position 0 — Bob Wilson	0x200007E4
Position 1 — John Doe	0x200008E0
Position 2 — Jane Smith	0x200009DC

Final results for all patients

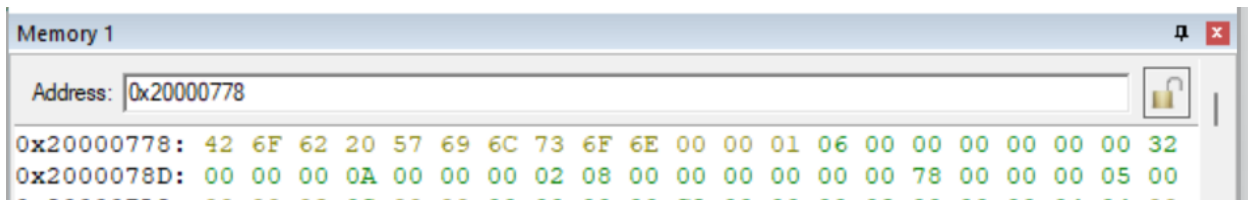
Patient 1 —

Bob Wilson (Most Critical)

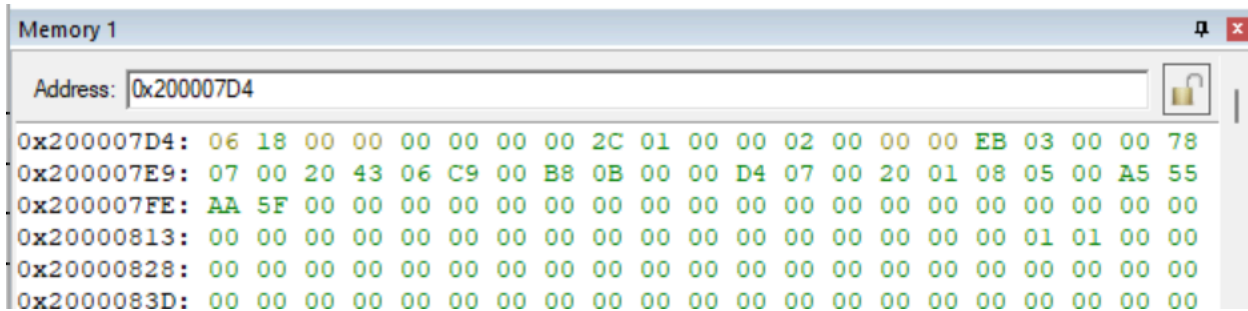
Base Address: 0x200007E4



name_ptr: 0x20000778



medicine_ptr_3: 0x200007D4



Basic Information

Offset	Field	Decimal	Hex
+0x00	patient_id	1003	0x03EB
+0x04	name_ptr	→ “Bob Wilson”	0x20000778
+0x08	age	67	0x43
+0x09	treatment_code	6	0x06
+0x0A	ward_number	201	0x00C9

Offset	Field	Decimal	Hex
+0x0C	room_daily_rate	3000	0x00000BB8
+0x10	medicine_list_ptr	"medicine_ptr_3"	0x200007D4
+0x14	medicine_count	1	0x01
+0x15	alert_count	8	0x08
+0x16	stay_days	5	0x0005

Vital Signs

Offset	Field	Decimal	Hex
+0x18	heart_rate	165	0xA5
+0x19	oxygen_level	85	0x55
+0x1A	systolic_bp	170	0xAA
+0x1B	diastolic_bp	95	0x5F

System Flags

Offset	Field	Decimal	Hex
+0x40	vital_buffer_index	1	0x01
+0x41	alert_flag	1	0x01
+0x42	dosage_due_flag	0/1	0x00 / 0x01

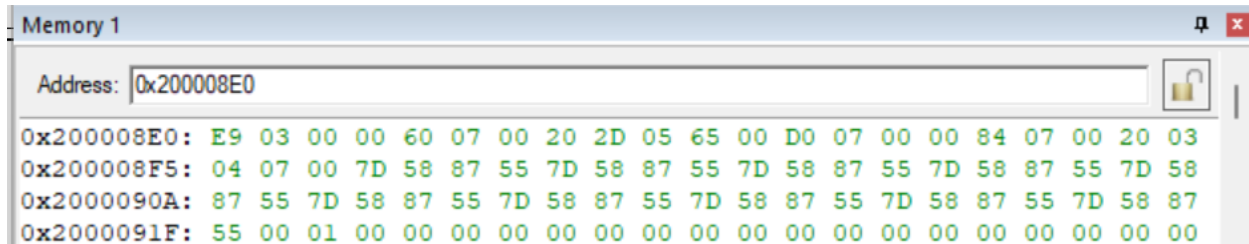
Billing

Offset	Field	Decimal	Hex
+0xE4	treatment_cost	30,000	0x00007530
+0xE8	room_cost	15,000	0x00003A98
+0xEC	medicine_cost	3,000	0x00000BB8
+0xF0	lab_test_cost	2,500	0x000009C4
+0xF4	total_bill	50,500	0x0000C544
+0xF8	overflow_flag	0	0x00

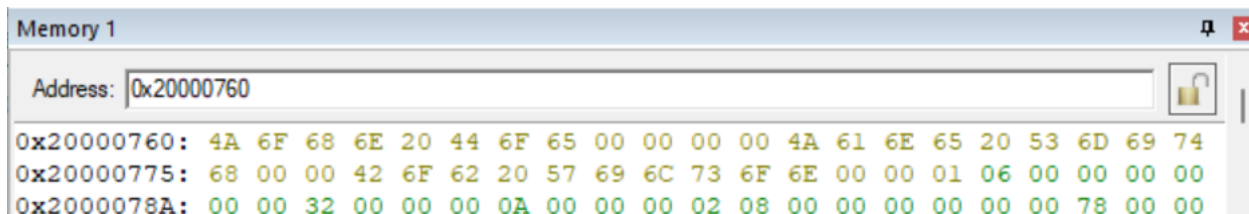
Patient 2 —

John Doe (Moderate Critical)

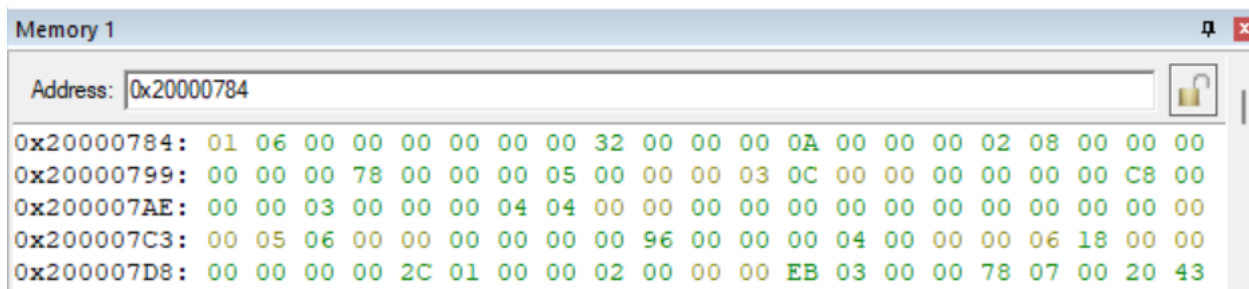
Base Address: 0x200008E0



name_ptr: 0x20000760



medicine_ptr_1: 0x20000784



Basic Information

Offset	Field	Decimal	Hex
+0x00	patient_id	1001	0x03E9
+0x04	name_ptr	→ "John Doe"	0x20000760
+0x08	age	45	0x2D
+0x09	treatment_code	5	0x05
+0x0A	ward_number	101	0x0065
+0x0C	room_daily_rate	2000	0x000007D0
+0x10	medicine_list_ptr	"medicine_ptr_1"	0x20000784
+0x14	medicine_count	3	0x03

Offset	Field	Decimal	Hex
+0x15	alert_count	4	0x04
+0x16	stay_days	7	0x0007

Vital Signs

Offset	Field	Decimal	Hex
+0x18	heart_rate	125	0x7D
+0x19	oxygen_level	88	0x58
+0x1A	systolic_bp	135	0x87
+0x1B	diastolic_bp	85	0x55

System Flags

Offset	Field	Decimal	Hex
+0x40	vital_buffer_index	0	0x00
+0x41	alert_flag	1	0x01
+0x42	dosage_due_flag	0/1	0x00 / 0x01

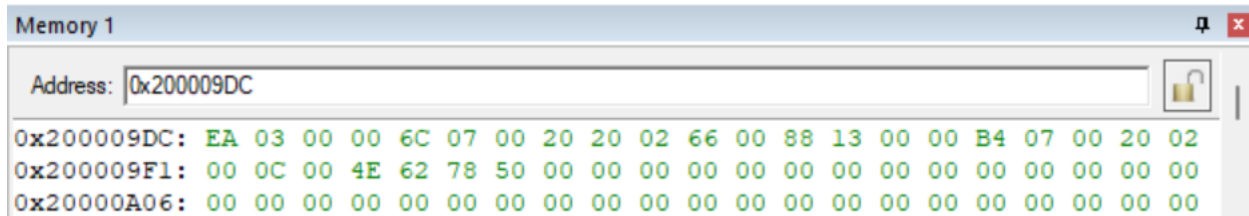
Billing

Offset	Field	Decimal	Hex
+0xE4	treatment_cost	25,000	0x000061A8
+0xE8	room_cost	14,000	0x000036B0
+0xEC	medicine_cost	11,900	0x00002E7C
+0xF0	lab_test_cost	3,000	0x00000BB8
+0xF4	total_bill	53,900	0x0000D28C
+0xF8	overflow_flag	0	0x00

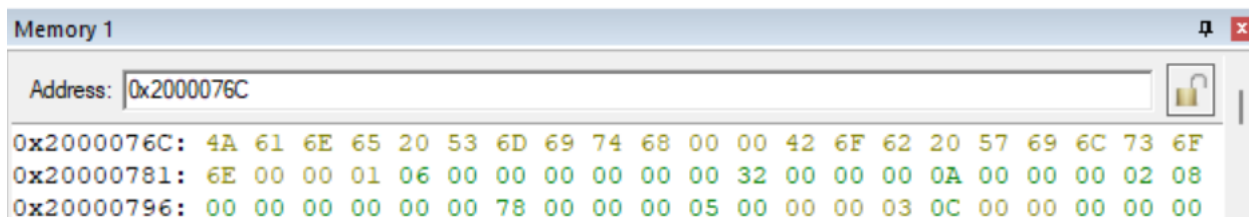
Patient 3 —

Jane Smith (Least Critical)

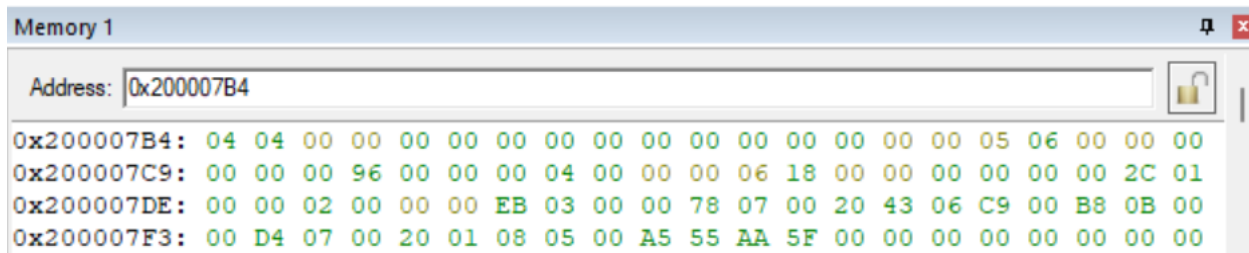
Base Address: 0x200009DC



name_ptr : 0x2000076C



medicine_ptr_2: 0x200007B4



Basic Information

Offset	Field	Decimal	Hex
+0x00	patient_id	1002	0x03EA
+0x04	name_ptr	→ “Jane Smith”	0x2000076C
+0x08	age	32	0x20
+0x09	treatment_code	2	0x02
+0x0A	ward_number	102	0x0066
+0x0C	room_daily_rate	5000	0x00001388
+0x10	medicine_list_ptr	"medicine_ptr_2"	0x200007B4
+0x14	medicine_count	2	0x02
+0x15	alert_count	0	0x00
+0x16	stay_days	12	0x000C

Vital Signs

Offset	Field	Decimal	Hex
+0x18	heart_rate	78	0x4E
+0x19	oxygen_level	98	0x62
+0x1A	systolic_bp	120	0x78
+0x1B	diastolic_bp	80	0x50

System Flags

Offset	Field	Decimal	Hex
+0x40	vital_buffer_index	1	0x01
+0x41	alert_flag	0	0x00
+0x42	dosage_due_flag	0/1	0x00 / 0x01

Billing

Offset	Field	Decimal	Hex
+0xE4	treatment_cost	50,000	0x0000C350
+0xE8	room_cost	57,000	0x0000DEA8
+0xEC	medicine_cost	7,200	0x00001C20
+0xF0	lab_test_cost	8,000	0x00001F40
+0xF4	total_bill	122,200	0x0001DD58
+0xF8	overflow_flag	0	0x00

Quick Memory Verifications

Sorted IDs:

0x200007E4 → EB 03 00 00 (1003 / 0x03EB)

0x200008E0 → E9 03 00 00 (1001 / 0x03E9)

0x200009DC → EA 03 00 00 (1002 / 0x03EA)

Alert Counts:

$0x200007E4 + 0x15 \rightarrow 08 \text{ (0x08)}$

$0x200008E0 + 0x15 \rightarrow 04 \text{ (0x04)}$

$0x200009DC + 0x15 \rightarrow 00 \text{ (0x00)}$

Total Bills:

$0x200007E4 + 0xF4 \rightarrow 44 \text{ C5 } 00 \text{ } 00 \text{ (50,500 / 0x0000C544)}$

$0x200008E0 + 0xF4 \rightarrow 8C \text{ D2 } 00 \text{ } 00 \text{ (53,900 / 0x0000D28C)}$

$0x200009DC + 0xF4 \rightarrow 58 \text{ DD } 01 \text{ } 00 \text{ (122,200 / 0x0001DD58)}$

Notes: Known Implementation Observations

1. **Module 10 vital field mapping:** prints BP and O2 using swapped offsets relative to how *Module 2* stores them.
2. **Alert_count initialization strategy:** `data.s` sets initial `alert_count`; *Module1* intentionally avoids overwriting it.
3. **Lab test cost persistence:** *Module1* does not clear `lab_test_cost`; values from data drive totals.
4. **Sensor malfunction early return:** check routine logs the first stuck sensor it finds (*HR* first), so it may not log *O2/SBP/DBP* even if stuck.

Limitations

Simulation Only

- UART via ITM (not real UART)
- Flash writes simulated in RAM

Fixed Patient Count

- Maximum 3 patients
- Fixed medicine list size

Error Log Capacity

- 20 records
- Circular overwrite

THE END